Machine Learning: A Comprehensive Textbook

Solutions to Chapter 3 Exercises

Data Preprocessing and Feature Engineering

Solutions Manual

Exercise 3.1 — When Mean Imputation Fails

Describe two scenarios where mean imputation would be inappropriate. Propose alternative strategies.

Solution. **Scenario 1: Missing-not-at-random (MNAR) data.** Suppose income is missing disproportionately for very high earners (who decline to disclose) or very low earners (survey non-response). Imputing with the overall mean pulls all missing values toward the center, destroying exactly the tail information that matters for e.g. a credit-risk model, and understates variance. *Better strategy:* add a binary "was-missing" indicator feature alongside the imputed value (so the model can learn that missingness itself is informative), or use a model-based imputation (e.g. MICE / iterative imputer, or a quantile/stratified imputation conditioned on other correlated features) that respects the conditional distribution rather than a single global mean.

Scenario 2: Multimodal or skewed features. Consider a feature like "time to complete a task" that is bimodal (fast automated submissions vs. slow manual ones). The mean can fall in the low-density valley between the two modes, so mean-imputed points look like a third, non-existent population and distort clustering/distance-based methods (e.g. k-NN, k-means). *Better strategy:* use the median (robust to skew) or, better, k-nearest-neighbor imputation / imputation by group (e.g. impute within the automated vs. manual subgroup if that's inferable from other features) so the imputed value falls within the correct mode.

In general, mean imputation is inappropriate whenever missingness is informative (MNAR) or the feature distribution is not unimodal/symmetric — both violate the implicit assumption that "the mean is a reasonable stand-in for any missing value."

Exercise 3.2 — Standardisation

Prove that standardisation $x \mapsto (x-\mu)/\sigma$ transforms the data so the standardised feature has mean 0 and variance 1.

Solution. Let $z^{(i)} = (x^{(i)} - \mu)/\sigma$ for $i = 1, \dots, n$, where $\mu = \frac{1}{n}\sum_i x^{(i)}$ and $\sigma^2 = \frac{1}{n}\sum_i (x^{(i)} - \mu)^2$.

Mean of z:

$$\bar{z} = \frac{1}{n}\sum_{i=1}^{n} z^{(i)} = \frac{1}{n}\sum_{i=1}^{n} \frac{x^{(i)} - \mu}{\sigma} = \frac{1}{\sigma}\left(\frac{1}{n}\sum_i x^{(i)} - \mu\right) = \frac{1}{\sigma}(\mu - \mu) = 0.$$

Variance of z:

$$\mathrm{Var}(z) = \frac{1}{n}\sum_{i=1}^{n}(z^{(i)} - \bar{z})^2 = \frac{1}{n}\sum_i (z^{(i)})^2 \quad (\text{since } \bar{z} = 0)$$

$$= \frac{1}{n}\sum_i \frac{(x^{(i)} - \mu)^2}{\sigma^2} = \frac{1}{\sigma^2}\cdot\frac{1}{n}\sum_i (x^{(i)} - \mu)^2 = \frac{\sigma^2}{\sigma^2} = 1.$$

So the standardised feature has exactly mean 0 and variance 1. ■

Exercise 3.3 — Train-Only Normalisation

Why must normalisation statistics be computed only on the training set? Construct a concrete example where full-dataset statistics lead to optimistic bias.

Solution. This is the same leakage principle as Exercise 1.7: normalisation statistics computed on the full dataset (train+test) let information about the test set's distribution influence the features seen during training, violating the premise that the test set represents genuinely unseen future data — a deployed model will only ever have access to training-time statistics when it standardises a brand-new incoming example.

Concrete example. Suppose a feature x is a sensor reading whose scale drifts slightly over time (e.g. due to sensor calibration decay), so early (training-period) samples have $x \sim \mathcal{N}(10, 2^2)$ and later (test-period) samples have $x \sim \mathcal{N}(13, 2^2)$ — a real, deployment-relevant covariate shift. If we standardise using only the training mean $\hat{\mu}_{tr} = 10$, the test features arrive shifted (centered around $(13-10)/2 = 1.5$ rather than 0), correctly exposing the shift to the evaluation, so the reported test accuracy reflects how the model actually degrades under drift.

If, instead, we standardise using the combined mean $\hat{\mu}_{all} \approx 11.5$, the test features are recentered to be roughly symmetric around 0 again — the pipeline has "absorbed" the very shift that would occur in production, since the true deployment-time mean of 13 is never known in advance. This produces optimistic test performance (the model appears robust to a shift that, honestly evaluated, it is not), because the leakage specifically cancels the population difference the held-out split was designed to reveal.

Exercise 3.4 — First Principal Component Derivation

Show that the first principal component u_1 is the eigenvector of the covariance matrix C with the largest eigenvalue, by solving $\max_{\|u\|=1} u^\top C u$.

Solution. We seek $u_1 = \arg\max_{\|u\|=1} u^\top C u$, where C is the (symmetric, positive semi-definite) sample covariance matrix. Use Lagrange multipliers to enforce the constraint $u^\top u = 1$:

$$\mathcal{L}(u, \lambda) = u^\top C u - \lambda(u^\top u - 1).$$

Setting the gradient with respect to u to zero:

$$\nabla_u \mathcal{L} = 2Cu - 2\lambda u = 0 \implies Cu = \lambda u.$$

This says any *critical point* u of the constrained optimisation must be a unit eigenvector of C, with the Lagrange multiplier λ equal to the corresponding eigenvalue. Evaluating the objective at such a critical point:

$$u^\top C u = u^\top(\lambda u) = \lambda(u^\top u) = \lambda.$$

So the objective value at any eigenvector-critical-point is exactly its eigenvalue λ. Since C is symmetric, it has an orthonormal eigenbasis $u_1, \ldots, u_d$ with eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$ (PSD $\Rightarrow$ eigenvalues ≥ 0). Among all these critical points, the maximum objective value λ_1 is attained at the eigenvector corresponding to the *largest* eigenvalue. Since every unit vector can be written in this eigenbasis as $u = \sum_j c_j u_j$ with $\sum_j c_j^2 = 1$, we have $u^\top C u = \sum_j c_j^2 \lambda_j \leq \lambda_1 \sum_j c_j^2 = \lambda_1$, with equality iff $u = u_1$ (or any unit vector in the top eigenspace) — confirming this is indeed the

global maximum, not just a critical point. Hence u_1, the leading eigenvector of C, maximizes $u^\top C u$ over the unit sphere, i.e. it is the direction of maximum variance — the first principal component. ■

Exercise 3.5 — PCA Numerical Example

Eigenvalues $[3.2, 1.5, 0.8, 0.3, 0.2]$ (1000 samples, 5 features). (a) PCs for 90% variance. (b) Scree plot description. (c) Reconstruction error with 2 PCs.

Solution. Total variance $= 3.2 + 1.5 + 0.8 + 0.3 + 0.2 = 6.0$.

(a) Cumulative variance explained:

- 1 PC: $3.2/6.0 = 53.3\%$
- 2 PCs: $(3.2 + 1.5)/6.0 = 4.7/6.0 = 78.3\%$
- 3 PCs: $(4.7 + 0.8)/6.0 = 5.5/6.0 = 91.7\%$

So $\boxed{3}$ principal components are needed to reach (and exceed) 90% explained variance (2 PCs only give 78.3%).

(b) Scree plot. A scree plot shows eigenvalue (y-axis) vs. component index (x-axis): points at $(1, 3.2), (2, 1.5), (3, 0.8), (4, 0.3), (5, 0.2)$, connected by a line. The plot shows a steep drop from PC1 to PC2 (a factor of $\sim 2.1\times$), a smaller drop from PC2 to PC3, and then a shallow "elbow"/tail from PC3 through PC5 where the remaining components contribute little (0.8, 0.3, 0.2) — the elbow at PC3 is the classical visual cue for choosing $k = 2$ or 3 retained components (the "elbow method"), consistent with the 90%-variance criterion in (a) landing at 3.

(c) Reconstruction error using 2 PCs. For PCA, the mean-squared reconstruction error equals the sum of the discarded eigenvalues (per-sample, averaged over the dataset, in the standardized-feature scale):

$$\text{Reconstruction error} = \sum_{j=3}^{5} \lambda_j = 0.8 + 0.3 + 0.2 = 1.3.$$

As a fraction of total variance: $1.3/6.0 \approx 21.7\%$ of the total variance is *unexplained* (lost) when retaining only 2 components — consistent with $100\% - 78.3\% = 21.7\%$ from part (a).

Exercise 3.6 — OHE vs. Target Encoding

Compare one-hot encoding and target encoding for a "country" feature with 195 unique values, $n = 10{,}000$. Discuss dimensionality, overfitting risk, computational cost.

Solution. **One-hot encoding (OHE).** Produces 195 new binary columns. *Dimensionality:* adds 194 columns beyond the original single feature — a large, sparse expansion, especially problematic if there are many other categorical features (dimensionality explosion, "curse of dimensionality" concerns for distance-based methods). *Overfitting risk:* generally low per-feature (no target leakage — OHE is purely a re-encoding of the category identity and carries no information about y), but rare countries (e.g. appearing only 1–2 times in 10,000 samples) get their own near-useless sparse column that a model can still overfit to (memorize a single training example's label via that column) in high-capacity models. *Computational cost:* moderate — 195 extra sparse columns is manageable for most models (tree-based or linear) at $n = 10{,}000$, though it does increase memory/compute roughly linearly in the cardinality.

Target encoding. Replaces each country with (a smoothed version of) the mean target value

for that country, keeping dimensionality at exactly 1 column. *Dimensionality:* no expansion — very compact, especially valuable when cardinality is very high (e.g. 10,000+ categories) where OHE would become impractical. *Overfitting risk:* substantially higher — for rare countries (few samples), the encoded value is essentially a memorized/near-noisy estimate of that country's own training labels, directly leaking target information into the feature; without careful regularisation (Bayesian smoothing toward the global mean, or out-of-fold/cross-validated encoding) this causes severe target leakage and optimistic training performance that does not generalise. *Computational cost:* very low (a single numeric column), but requires careful implementation (e.g. K-fold target encoding) to avoid leakage, adding pipeline complexity.

Recommendation. With 195 categories and $n = 10{,}000$ (roughly 51 samples/category on average, but likely a long tail of rare countries), OHE is safer if using linear/tree models robust to moderate sparsity; target encoding is preferable if cardinality were much higher (e.g. zip codes) but should always be computed out-of-fold (fit on training folds, applied to validation fold) to control leakage. A middle ground is frequency encoding or hashing, or grouping rare countries into an "other" bucket before OHE.

Exercise 3.7 — Extreme Imbalance

Fraud dataset, 0.1% positive rate. (a) Majority-classifier accuracy. (b) Why is accuracy bad here? (c) Two approaches to imbalance.

Solution. **(a)** A classifier that always predicts the majority (non-fraud) class is correct on exactly the 99.9% of samples that are truly non-fraud, so its accuracy is $\boxed{99.9\%}$.

(b) This 99.9% accuracy is achieved while catching *zero* frauds (recall = 0 on the class that actually matters). Accuracy weights every sample equally, so with a 999:1 class ratio, the metric is completely dominated by performance on the majority class and is essentially blind to how well the model identifies the rare, business-critical positive class — a model can be "highly accurate" and simultaneously useless for the actual task (identical failure mode to Exercises 1.1 and 1.6).

(c) Two approaches:

1. **Resampling.** Oversample the minority class (e.g. SMOTE — synthetic minority oversampling, which interpolates between nearby minority examples in feature space) or undersample the majority class, to rebalance the class ratio seen during training so the loss function is not overwhelmed by majority-class gradient contributions. Trade-off: oversampling risks overfitting to synthetic/duplicated minority points; undersampling discards potentially useful majority-class data.
2. **Cost-sensitive / reweighted learning.** Keep the original data but weight the loss so minority-class errors are penalized more (e.g. $\mathcal{L} = \sum_i w_{y_i}\, \ell(y_i, \hat{y}_i)$ with $w_1 \gg w_0$, or use 'class_weight="balanced"' in scikit-learn), directly addressing the imbalance in the optimization objective without altering the data itself. This is often preferable when synthetic samples are undesirable, and it composes naturally with the asymmetric-cost decision-threshold ideas from Exercise 1.6.

Exercise 3.8 — Programming: PCA on Breast Cancer Dataset

Apply PCA to `sklearn.datasets.load_breast_cancer`, plot in 2D colour-coded by class. How well does PCA separate the classes?

Solution. **Reference implementation.**

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

data = load_breast_cancer()
X, y = data.data, data.target

X_scaled = StandardScaler().fit_transform(X)   # standardise first! (Ex. 3.2)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(6,5))
plt.scatter(X_pca[y==0,0], X_pca[y==0,1], label='malignant', alpha=0.6)
plt.scatter(X_pca[y==1,0], X_pca[y==1,1], label='benign', alpha=0.6)
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% var)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% var)')
plt.legend(); plt.title('Breast Cancer Wisconsin: PCA projection')
plt.show()
```

Expected result. Standardising first is essential here because the 30 raw features are on wildly different scales (e.g. "mean area" is in the hundreds while "mean smoothness" is ~ 0.1); without standardising, PCA is dominated by the highest-variance raw feature. With standardisation, the first two principal components together typically explain roughly 60–65% of the total variance for this dataset, and the resulting 2D scatter plot shows the malignant and benign classes forming two largely separated (though not perfectly linearly separable) clusters along PC1, which strongly correlates with overall cell nucleus size/shape irregularity. A small region of overlap remains between the classes near the cluster boundary, reflecting the intrinsically harder borderline cases; nonetheless PCA to just 2 dimensions preserves the great majority of the discriminative signal present in the original 30 features, illustrating that a small number of principal components can capture most of the class-relevant structure for this dataset even though PCA itself is unsupervised (it does not use the labels y at all when computing components — the separation observed is an emergent property of the data's covariance structure aligning with the class distinction).